

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет безопасности информационных технологий

Дисциплина:
«Основы системного программирования»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №1
«Работа с файлами и каталогами»

Выполнил:

Пахомов Владимир Юрьевич, студент группы N3250



(подпись)

Проверил:

Грозов Владимир Андреевич

(отметка о выполнении)

(подпись)

Санкт-Петербург

2026.

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ.....	2
ВВЕДЕНИЕ.....	3
ХОД РАБОТЫ.....	3

ЗАДАНИЕ. ВАРИАНТ 1

Таблица 1:

-v, --version - Вывод версии программы и информации о программе (ФИО исполнителя, номер группы, номер варианта лабораторной).

-h, --help - Вывод справки по опциям.

Таблица 2:

LAB1DEBUG - Включение вывода отладочной информации.

Таблица 3:

Количество букв в **фамилии** студента, выполняющего работу, **нечетное**.

Выполняется поиск заданной последовательности байтов. Аргумент `цель_поиска` имеет формат `0xhh[hh*]`, где `hh` — две шестнадцатеричных цифры (число должно начинаться префикса `0x`).

Пример запуска программы:

```
./lab1psiN32451 /home 0xc0ffee
```

(`open/getdents64`, без рекурсии, поиск HEX в формате `0xhh[hh*]`)

Таблица 4:

Вариант 1 - `open()/getdents64()` без рекурсии

ХОД РАБОТЫ

Исходный код Makefile:

```
1 | CC = gcc
2 | CFLAGS = -O3 -Wall -Wextra -Werror -fanalyzer
3 | TARGET = lab1vypN3250
4 |
5 | all: $(TARGET)
6 |
7 | $(TARGET): lab1vypN3250.c
8 |     $(CC) $(CFLAGS) $(TARGET).c -o $(TARGET)
9 |
10 | clean:
11 |     rm -f $(TARGET)
12 |     rm -rf test
13 |
14 |
15 | test: all
16 |     @echo "--- Подготовка тестового дерева ---"
17 |     @rm -rf test && mkdir -p test/dir1/subdir test/dir2 test/empty_dir
18 |     @# Создаем файлы с тестовыми данными
19 |     @printf "\xDE\xAD\xBE\xEF" > test/dir1/hit1.bin
20 |     @printf "Обычный текст с \xCA\xFE\xBA\xBE внутри" > test/dir1/subdir/hit2.txt
21 |     @printf "\xDE\xAD\xBE\xEF и еще раз \xDE\xAD\xBE\xEF" > test/dir2/multiple.bin
22 |     @touch test/empty.file
23 |     @echo "Просто текст без совпадений" > test/no_match.txt
24 |
25 |     @echo ""
26 |     @echo "--- Тест 1: Базовый поиск ---"
27 |     ./$(TARGET) test 0xcafebabe
28 |
29 |     @echo ""
30 |     @echo "--- Тест 2: Поиск в глубоких подкаталогах ---"
31 |     ./$(TARGET) test 0xdeadbeef
32 |
33 |     @echo ""
34 |     @echo "--- Тест 3: Режим отладки (LAB1DEBUG) ---"
35 |     LAB1DEBUG=1 ./$(TARGET) test/dir2 0xdeadbeef
36 |
37 |     @echo ""
38 |     @echo "--- Тест 4: Поиск в пустом каталоге / отсутствие совпадений ---"
39 |     ./$(TARGET) test/empty_dir 0x123456
40 |
41 |     @echo ""
42 |     @echo "--- Тест 5: Обработка ошибок ---"
43 |     ./$(TARGET) ./non_existent_path 0x112233
44 |
45 |     @echo ""
46 |     @echo "--- Все тесты завершены ---"
```

Отчет valgrind со строки HEAP SUMMARY:

```
==49683== HEAP SUMMARY:
==49683==      in use at exit: 0 bytes in 0 blocks
==49683==    total heap usage: 13 allocs, 13 frees, 28,789 bytes allocated
==49683==
==49683== All heap blocks were freed -- no leaks are possible
==49683==
==49683== For lists of detected and suppressed errors, rerun with: -s
==49683== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```



```

• vladimir@vladimirp ~/Desktop/ОСП/lab1vypN3250 ./lab1vypN3250 -v
Пахомов Владимир Юрьевич, гр. N3250
Вариант: 1
• vladimir@vladimirp ~/Desktop/ОСП/lab1vypN3250 ./lab1vypN3250 -h
Использование: lab1vypN3250 [опции] <каталог> <цель_поиска>
Опции:
  -v, --version  Вывод версии и информации об авторе
  -h, --help     Вывод этой справки

⊙ vladimir@vladimirp ~/Desktop/ОСП/lab1vypN3250 ./lab1vypN3250 -g
./lab1vypN3250: invalid option -- 'g'
Введены неизвестные опции. Используйте -h для справки
⊙ ✗ vladimir@vladimirp ~/Desktop/ОСП/lab1vypN3250 ./lab1vypN3250
Ошибка: Необходимо ввести название каталога и цель поиска
Использование: lab1vypN3250 [опции] <каталог> <цель_поиска>
Опции:
  -v, --version  Вывод версии и информации об авторе
  -h, --help     Вывод этой справки

• ✗ vladimir@vladimirp ~/Desktop/ОСП/lab1vypN3250 ./lab1vypN3250 test 0xDE
test/dir2/multiple.bin
test/dir1/hit1.bin
⊙ vladimir@vladimirp ~/Desktop/ОСП/lab1vypN3250 ./lab1vypN3250 test 0xDE5
Ошибка: Неверный формат HEX
⊙ ✗ vladimir@vladimirp ~/Desktop/ОСП/lab1vypN3250 ./lab1vypN3250 test DE5
Ошибка: Неверный формат HEX
⊙ ✗ vladimir@vladimirp ~/Desktop/ОСП/lab1vypN3250

```

Рисунок 1: Пример работы программы

```
• x vladimir@vladimirp ~/Desktop/ОСП/lab1vypN3250 make test
--- Подготовка тестового дерева ---

--- Тест 1: Базовый поиск ---
./lab1vypN3250 test 0xcafebabe
test/dir1/subdir/hit2.txt

--- Тест 2: Поиск в глубоких подкаталогах ---
./lab1vypN3250 test 0xdeadbeef
test/dir2/multiple.bin
test/dir1/hit1.bin

--- Тест 3: Режим отладки (LAB1DEBUG) ---
LAB1DEBUG=1 ./lab1vypN3250 test/dir2 0xdeadbeef
Включен вывод отладочных сообщений
Файл test/dir2/multiple.bin: Найдено совпадение 0
test/dir2/multiple.bin

--- Тест 4: Поиск в пустом каталоге / отсутствие совпадений ---
./lab1vypN3250 test/empty_dir 0x123456

--- Тест 5: Обработка ошибок ---
./lab1vypN3250 ./non_existent_path 0x112233
Ошибка доступа к каталогу ./non_existent_path: No such file or directory

--- Все тесты завершены ---
○ vladimir@vladimirp ~/Desktop/ОСП/lab1vypN3250
```

Рисунок 2: Результат работы тестов из Makefile

Исходный код программы lab1vypN3250:

```
1  | #include <errno.h>
2  | #include <stdio.h>
3  | #include <stdlib.h>
4  | #include <string.h>
5  | #include <getopt.h>
6  | #include <sys/syscall.h>
7  | #include <unistd.h>
8  | #include <fcntl.h>
9  | #include <dirent.h>
10 |
11 |
12 | #define MAX_PATH 4096
13 | #define BUFFER_SIZE 4096
14 |
15 |
16 | struct linux_dirent64 {
17 |     unsigned long long d_ino;
18 |     long long          d_off;
19 |     unsigned short      d_reclen;
20 |     unsigned char       d_type;
21 |     char                d_name[];
22 | };
23 |
24 |
25 | typedef struct StackNode {
26 |     char path[MAX_PATH];
27 |     struct StackNode *next;
28 | } StackNode;
29 |
30 | void push(StackNode **top, const char *path) {
31 |     StackNode *node = malloc(sizeof(StackNode));
32 |     if (!node) {
33 |         perror("malloc");
34 |         exit(1);
35 |     }
36 |     strncpy(node->path, path, MAX_PATH - 1);
37 |     node->path[MAX_PATH - 1] = '\0';
38 |
39 |     node->next = *top;
40 |     *top = node;
41 | }
42 |
43 | char *pop(StackNode **top) {
44 |     if (*top == NULL) return NULL;
45 |     StackNode *node = *top;
46 |     char *path = strdup(node->path);
47 |     if (!path) {
48 |         perror("strdup");
49 |         exit(1);
50 |     }
51 |     *top = node->next;
52 |     free(node);
53 |     return path;
54 | }
55 |
56 | int hexToBytes(const char *hexStr, unsigned char *bytes) {
57 |     if (strncmp(hexStr, "0x", 2) != 0) return -1;
58 |     const char *p = hexStr + 2;
59 |     int length = strlen(p);
60 |     if (length == 0 || length % 2 != 0) return -1;
61 |
62 |     int byteCount = 0;
63 |     for (int i = 0; i < length; i += 2) {
64 |         unsigned int temp;
65 |         if (sscanf(p + i, "%2x", &temp) != 1) return -1;
66 |         bytes[byteCount++] = (unsigned char)temp;
67 |     }
68 |     return byteCount;
69 | }
70 |
71 | int search(const char *filepath, unsigned char *target, int targetLength, int debug) {
```



```

72 |     int fd = open(filepath, O_RDONLY);
73 |     if (fd < 0) return 0;
74 |
75 |     unsigned char buffer[BUFFER_SIZE];
76 |     ssize_t n;
77 |     int found = 0;
78 |     long totalOffset = 0;
79 |
80 |     while ((n = read(fd, buffer, BUFFER_SIZE)) > 0) {
81 |         for (int i = 0; i <= n - targetLength; i++) {
82 |             if (memcmp(&buffer[i], target, targetLength) == 0) {
83 |                 if (debug) fprintf(stderr, "Файл %s: Найдено совпадение %ld\n", filepath,
totalOffset + i);
84 |                 found = 1;
85 |                 break;
86 |             }
87 |         }
88 |         if (found) break;
89 |         totalOffset += n;
90 |         if (n == BUFFER_SIZE) {
91 |             lseek(fd, -(targetLength - 1), SEEK_CUR);
92 |             totalOffset -= (targetLength - 1);
93 |         }
94 |     }
95 |     close(fd);
96 |     return found;
97 | }
98 |
99 | int main(int argc, char* argv[]) {
100 |     char *DEBUG = getenv("LAB1DEBUG");
101 |     int debugMode = 0;
102 |     if (DEBUG) {
103 |         fprintf(stderr, "Включен вывод отладочных сообщений\n");
104 |         debugMode = 1;
105 |     }
106 |
107 |     int opt;
108 |     char *author = "Пахомов Владимир Юрьевич, гр. N3250\nВариант: 1";
109 |     char *help = "Использование: lab1vupN3250 [опции] <каталог> <цель_поиска>\nОпции:\n  -v,
--version Вывод версии и информации об авторе\n  -h, --help Вывод этой справки\n";
110 |
111 |
112 |     static struct option longopts[] = {
113 |         {"version", no_argument, 0, 'v'},
114 |         {"help", no_argument, 0, 'h'},
115 |         {0, 0, 0, 0},
116 |     };
117 |
118 |     while ((opt = getopt_long(argc, argv, "vh", longopts, NULL)) != -1) {
119 |         if (opt == 'v') {
120 |             printf("%s\n", author);
121 |             return 0;
122 |         }
123 |         else if (opt == 'h') {
124 |             printf("%s\n", help);
125 |             return 0;
126 |         }
127 |         else {
128 |             fprintf(stderr, "Введены неизвестные опции. Используйте -h для справки\n");
129 |             return 1;
130 |         }
131 |     }
132 |
133 |     if (argc - optind < 2) {
134 |         fprintf(stderr, "Ошибка: Необходимо ввести название каталога и цель поиска\n");
135 |         printf("%s\n", help);
136 |         return 1;
137 |     }
138 |
139 |     char *searchDir = argv[optind++];
140 |     char *searchString = argv[optind];
141 |
142 |     unsigned char targetBytes[256];
143 |     int targetLen = hexToBytes(searchString, targetBytes);

```

```

144 |         if (targetLen <= 0) {
145 |             fprintf(stderr, "Ошибка: Неверный формат HEX\n");
146 |             return 1;
147 |         }
148 |
149 |         StackNode *dirsStack = NULL;
150 |         push(&dirsStack, searchDir);
151 |
152 |         while (dirsStack != NULL) {
153 |             char *currentDir = pop(&dirsStack);
154 |             int fd = open(currentDir, O_RDONLY | O_DIRECTORY);
155 |
156 |             if (fd == -1) {
157 |                 fprintf(stderr, "Ошибка доступа к каталогу %s: %s\n", currentDir,
strerror(errno));
158 |                 free(currentDir);
159 |                 continue;
160 |             }
161 |
162 |             char buffer[BUFFER_SIZE];
163 |             int nread;
164 |             while ((nread = syscall(SYS_getdents64, fd, buffer, BUFFER_SIZE)) > 0) {
165 |                 for (int bpos = 0; bpos < nread; ) {
166 |                     struct linux_dirent64 *d = (struct linux_dirent64 *) (buffer + bpos);
167 |
168 |                     if (strcmp(d->d_name, ".") == 0 || strcmp(d->d_name, "..") == 0) {
169 |                         bpos += d->d_reclen;
170 |                         continue;
171 |                     }
172 |
173 |                     char fullPath[MAX_PATH];
174 |                     snprintf(fullPath, MAX_PATH, "%s/%s", currentDir, d->d_name);
175 |
176 |                     if (d->d_type == DT_DIR) {
177 |                         push(&dirsStack, fullPath);
178 |                     } else if (d->d_type == DT_REG) {
179 |                         if (search(fullPath, targetBytes, targetLen, debugMode)) {
180 |                             printf("%s\n", fullPath);
181 |                         }
182 |                     }
183 |
184 |                     bpos += d->d_reclen;
185 |                 }
186 |             }
187 |             close(fd);
188 |             free(currentDir);
189 |         }
190 |
191 |         return 0;
192 |     }
193 | }

```